

MANUAL COMPLETO

ForjaIA

Forja local self-hosted de software: pipeline multiagente
(Arquiteto → Codificador → QA → Segurança → Depurador →
Curador → DevOps → Deploy → Teste Humano → Corretor →
Checklist de Produção → Relatório), com execuções
persistentes, autenticação, RBAC por papel e isolamento de
workspace.

Manual do Usuário e Manual do Desenvolvedor · versão referente ao estado do repositório em 29/08/2026 (35 ADRs registrados)

Parte 1 — Manual do Usuário

1. O que é o ForjaIA
2. Instalação e primeiro acesso
3. Como funciona uma forja, passo a passo
4. A interface, painel a painel
5. Papéis de equipe e aprovação
6. Deploy e teste humano
7. Dogfooding automático
8. Backup e continuidade
9. Perguntas frequentes

Parte 2 — Manual do Desenvolvedor

10. Arquitetura em uma página

11. Estrutura de pastas
12. O pipeline de agentes em detalhe
13. API REST completa
14. WebSocket e eventos ao vivo
15. Banco de dados
16. Provedores de LLM e fallback
17. RBAC e equipe
18. Variáveis de ambiente
19. Scripts npm
20. Testes
21. Deploy multiplataforma
22. Dogfooding automático (implementação)
23. Decisões arquiteturais (ADRs)
24. Como estender o pipeline

Parte 1 — Manual do Usuário

Para quem vai **pedir** uma forja, acompanhar o pipeline e aprovar etapas — não é preciso ler código para usar esta parte.

1. O que é o ForjaIA

ForjaIA é uma "forja" de software: você descreve o que quer construir numa frase (a **Ordem**), e um pipeline de agentes de IA — cada um especializado numa etapa — projeta, escreve, testa, audita a segurança, corrige, sobe num ambiente real (Docker ou, para apps mobile, Simulador/Emulador) e testa como um usuário faria, até chegar num app funcional com relatório em PDF.

Diferente de "pedir código pra um chat", o ForjaIA roda tudo **localmente** (self-hosted — seus segredos de API nunca saem do seu servidor), guarda o histórico completo de cada execução (logs, versões de arquivo, testes, achados de segurança) e **para em pontos definidos** pedindo aprovação humana antes de seguir — nada acontece por trás das suas costas.

2. Instalação e primeiro acesso

Requisitos

- Node.js 20 ou mais recente
- Docker Desktop/Engine (obrigatório quando `FORJA_REQUIRE_DOCKER=true`, o padrão em produção)
- Uma chave de API de LLM (Gemini, Claude ou OpenAI) **ou** Ollama rodando localmente

Passo a passo (desenvolvimento)

```
cp .env.example .env
# define FORJA_API_TOKEN e a chave do provedor de LLM escolhido (ou use Ollama, sem chave)

npm install
npm install --prefix backend
npm install --prefix frontend

npm run dev
```

Abra `http://localhost:5173`. A API sobe em `http://127.0.0.1:3001`.

Execução local "de produção" (recomendado no dia a dia)

```
cp .env.example .env
# ajuste FORJA_API_TOKEN, FORJA_REQUIRE_DOCKER=true e a chave de LLM

npm run start:local-prod
```

Um único processo (API + interface estática) em `http://127.0.0.1:3001`, com Docker obrigatório, sem respostas simuladas ("mock") e só ouvindo na própria máquina (*loopback*).

Primeiro login

Na primeira tela, cole o `FORJA_API_TOKEN` definido no `.env` (ou o valor gerado automaticamente em `backend/data/.api-token`, se você não definiu um). Esse token identifica o usuário **admin** — quem tem esse token pode fazer qualquer coisa no sistema, inclusive criar outros membros de equipe com papéis mais restritos (ver seção 5).

3. Como funciona uma forja, passo a passo

Todo o pipeline é "**gated**": depois de cada etapa terminar (com sucesso ou falha), a execução para e espera alguém aprovar a próxima antes de continuar. Isso significa que você sempre pode revisar o que foi feito antes de deixar seguir — nenhuma etapa encadeia automaticamente na próxima sem aprovação.

```
Arquiteto → (aprovar) → Codificador → (aprovar) → QA → (aprovar) → Segurança → (aprovar) → [se algo falhou: Depurador → Curador → volta pro QA, até 3 tentativas] → (aprovar) → DevOps (carga/caos) → (aprovar) → Deploy → (aprovar) → Teste Humano → (aprovar) → [se falhou: Corretor → volta pro QA] → Checklist de Produção → (aprovar) → Relatório PDF
```

1. **Arquiteto** — lê sua Ordem e produz um plano: lista de arquivos a criar e, quando relevante, ADRs (decisões arquiteturais) para justificar escolhas não óbvias.
2. **Codificador** — escreve o conteúdo de cada arquivo do plano aprovado.
3. **QA** — gera e roda uma suíte de testes automatizados contra o código gerado.
4. **Segurança** — analisa o código em busca de vulnerabilidades. Se QA ou Segurança encontraram problema, o pipeline entra automaticamente no ciclo de cura (próximo item) antes de seguir — até 3 tentativas.
5. **Depurador** → **Curador** (só quando há falha) — o Depurador investiga a causa raiz sem mexer no código; o Curador aplica a correção com base nesse diagnóstico, e o fluxo volta pro QA para reconfirmar. Na última tentativa, o Curador troca de provedor de IA automaticamente (evita repetir um raciocínio que já falhou duas vezes).
6. **DevOps** — sobe o app numa sandbox Docker isolada e mede desempenho sob carga, incluindo testes de caos (falha de rede/CPU simulada ou real).
7. **Deploy** — publica o app de verdade: localmente via Docker (apps web) ou, para apps mobile Expo/React Native, no Simulador iOS e Emulador Android sempre, e em macOS/Windows quando o projeto já tem esse suporte configurado.
8. **Teste Humano** — simula um usuário real usando o app no ambiente publicado. Se encontrar problema, aciona o Corretor (próximo item); se passar, segue para o checklist final.
9. **Corretor** — aplica correções com base no relato do Teste Humano (ou num relato que você mesmo mandar a qualquer momento pelo Terminal — veja seção 4) e reabre o ciclo QA→Segurança.
10. **Checklist de Produção** — confere se os artefatos de produção existem (Dockerfile, `.env.example` etc.), completando o que faltar, e publica a release.
11. **Relatório** — gera um PDF final consolidando testes, achados de segurança, métricas de desempenho, diagnóstico e ADRs.

Você pode enviar um relato de problema para o Corretor a qualquer momento (não só na etapa de Teste Humano) — é o campo de chat no Terminal ("Fale com o Corretor"). Isso pausa a run correspondente na etapa de correção assim que possível.

4. A interface, painel a painel

Coluna esquerda

- **Ordem** — onde você descreve o que quer forjar, escolhe o projeto/destino de workspace, o ambiente (local/staging) e, opcionalmente, um teto de orçamento estimado em USD (a run pausa pedindo aprovação se o gasto estimado passar disso).
- **Pipeline** — trilho visual com todas as etapas, coloridas por estado (aguardando, ativa, aprovada, falhou), com a duração de cada uma.

Coluna central (abas)

- **Terminal** — log ao vivo de tudo que os agentes estão fazendo, e o campo para falar com o Corretor.
- **Código** — os arquivos gerados, com diff entre versões.
- **Segurança** — achados de segurança por severidade.
- **Diagnóstico** — o que o Depurador concluiu sobre a causa de uma falha.
- **Métricas** — RPS, latência, taxa de sucesso e o que aconteceu nos testes de caos.
- **Tokens** — consumo/custo de LLM por provedor.
- **ADRs** — as decisões arquiteturais do plano atual (editáveis antes de aprovar o Codificador).
- **Histórico** — runs anteriores, com exportação e PDF.
- **Auditoria** — auditorias independentes (Semgrep + `npm audit`, fora do pipeline principal) e o status do dogfooding automático agendado (seção 7).
- **Projetos** — projetos registrados no workspace.
- **Equipe** — gestão de membros e o quadro de runs (fila, aguardando aprovação, recentes) — só visível/editável conforme seu papel.

Coluna direita

Deploy (URL/estado do ambiente publicado), **Testes** (resumo aprovado/reprovado da run atual), **LLM & Tokens** (provedor/modelo em uso, com seleção manual) e **Regras** (estilo de código que o "Engenheiro Sênior" deve seguir).

Cabeçalho

Mostra seu papel, uma fileira de pontos de status de infraestrutura (API, Docker, Ollama, Cursor, WebSocket, dogfooding agendado — verde/vermelho/apagado conforme o estado), o modelo de IA em uso agora, e botões para controlar o serviço local (iniciar/reiniciar/parar/auto-restart).

5. Papéis de equipe e aprovação

Cada membro da equipe tem um papel, e cada etapa do pipeline só pode ser aprovada por papéis específicos — isso evita que, por exemplo, alguém sem contexto de segurança aprove sozinho uma etapa de Deploy.

Papel	Pode aprovar	Observação
admin	qualquer etapa	identidade fixa ligada ao <code>FORJA_API_TOKEN</code> ; único que cria/desativa membros

lead	qualquer etapa	gerencia serviços (start/stop/restart)
qa	QA, Segurança, Teste Humano	
sre	DevOps, Deploy, Checklist de Produção, Relatório	gerencia serviços
member	—	pode iniciar/cancelar runs e reportar problemas, mas não aprova etapas (a menos que também seja admin)
viewer	—	só leitura: nem inicia run, nem cancela, nem reporta problema

6. Deploy e teste humano

Para apps web, o deploy roda localmente via Docker. Para apps mobile (Expo/React Native), o ForjaIA tenta publicar em até 4 alvos, cada um independente dos outros (uma falha não derruba os demais):

- **Simulador iOS e Emulador Android** — sempre tentados.
- **macOS (Catalyst)** — só se o projeto já tiver esse suporte configurado.
- **Windows** — via GitHub Actions, só se o projeto já tiver o workflow configurado.

O Teste Humano usa o app real publicado — navegador de verdade (via Playwright) para web, e Appium/XCUITest para mobile — não é uma simulação de leitura de código.

7. Dogfooding automático

Toda segunda-feira às 6h, o próprio ForjaIA dispara uma forja real contra si mesmo (um app CRUD simples), aprova cada etapa sozinho e grava um relatório — sem precisar de ninguém acompanhando a tela. É como o ForjaIA testa a si mesmo continuamente, e foi assim que se encontrou um bug real de QA numa sessão anterior (ver ADR-034). O estado desse agendamento (ativo/inativo, resultado da última rodada) aparece no ponto de status do cabeçalho e num cartão na aba Auditoria. Detalhes de implementação na seção 22.

8. Backup e continuidade

O banco SQLite (projetos, runs, eventos, versões de arquivo) pode ser copiado com `npm run backup:db`, que mantém as últimas 14 cópias por padrão (ajustável). Se o processo do ForjaIA cair no meio de uma run, ele recupera o estado exato ao reiniciar (a run volta pausada na mesma etapa em que estava) — não é preciso recomeçar do zero.

9. Perguntas frequentes

Situação	O que fazer
Não sei meu token de acesso	Veja <code>FORJA_API_TOKEN</code> no <code>.env</code> , ou <code>backend/data/.api-token</code> se não foi definido manualmente
Botão de aprovar etapa está desabilitado	Seu papel não pode aprovar essa etapa específica — veja a tabela da seção 5

QA reprova um app que parece funcionalmente correto	Era um bug real e conhecido (ADR-034), já corrigido — se acontecer de novo, é um achado válido para reportar
Deploy mobile só publicou iOS/Android, não macOS/Windows	Comportamento esperado — esses dois exigem suporte já configurado no projeto (ver seção 6)
Serviço não sobe	<code>npm run service:status</code> pra diagnosticar; <code>npm run service:restart</code> pra reiniciar

Parte 2 — Manual do Desenvolvedor

Para quem vai mexer no código do próprio ForjaIA — arquitetura, API, banco, testes e como estender o pipeline.

10. Arquitetura em uma página

Peça	Função
Plano de controle	Express + WebSocket em <code>127.0.0.1</code> , autenticação Bearer, rate limiting (helmet)
SQLite (<code>better-sqlite3</code>)	Projetos, execuções (runs), eventos de log, versões de arquivo, preferências, equipe
Orquestrador (<code>agent/orchestrator.js</code>)	Máquina de estados do pipeline — decide a próxima etapa, gerencia pausas de aprovação, persiste progresso
Provedor LLM	Gemini, Claude, OpenAI, Ollama ou Cursor Agent (CLI, opt-in), com fallback e cooldown por billing
Raiz do workspace	Navegação e deploy restritos a <code>FORJA_WORKSPACE_ROOT</code> (sem <i>path traversal</i>)
Sandbox	Container Docker isolado com limites de memória/CPU, usado por QA/Segurança/DevOps
Frontend	React + Vite + TypeScript, WebSocket para eventos ao vivo, polling para status de infraestrutura

11. Estrutura de pastas

```
backend/  
  server.js           # todas as rotas HTTP + WebSocket  
  agent/  
    orchestrator.js   # máquina de estados do pipeline  
    architect.js, coder.js, qa.js, security.js, debugger.js,  
    healer.js, devops.js, human.js, userFix.js, reporter.js  
    stages/*.js       # wrappers finos por etapa (decidem o próximo pauseForApproval)  
  lib/  
    config.js, llm.js, rbac.js, team.js, seniorEngineer.js,  
    dbBackup.js, opsHealth.js, dogfoodStatus.js, productionChecklist.js,  
    mobileDeploy.js, androidDeploy.js, windowsDeploy.js, mobileHumanTest.js, ...  
  sandbox/  
  test/               # node --test, um arquivo por módulo  
frontend/src/  
  components/  
  components/tabs/  
  hooks/useForjaApp.ts # hook central de estado (a maior parte do estado da UI vive aqui)  
  services/api.ts, ws.ts # cliente HTTP e WebSocket  
docs/adr/  
scripts/  
  # backup-db.js, dogfood-forge.js, forja-service.js, check-local-prod.js
```

12. O pipeline de agentes em detalhe

O orquestrador (`agent/orchestrator.js`) executa cada etapa e, ao final de **todas** elas — sucesso ou falha — chama `pauseForApproval(nextStage, message)`, que grava `status: 'awaiting_approval'` e transmite o

evento `task-awaiting-approval` via WebSocket. Nada avança sem uma chamada explícita a `POST /api/agent/approve`, que valida RBAC (`assertCanApprove`) antes de liberar.

Etapa	Módulo	O que faz	Próxima etapa (gate)
Arquiteto	<code>agent/architect.js</code>	Gera plano de arquivos + ADRs a partir do prompt	<code>coder</code>
Codificador	<code>agent/coder.js</code> via <code>stages/coderStage.js</code>	Escreve o conteúdo de todos os arquivos do plano	<code>qa</code>
QA	<code>agent/qa.js</code> via <code>stages/qaStage.js</code>	Gera e roda suíte de testes contra os arquivos	<code>security</code> (sempre)
Segurança	<code>agent/security.js</code> via <code>stages/securityStage.js</code>	Análise estática/dinâmica de segurança	<code>debugger</code> se falhou e ainda há tentativas de cura; senão <code>devops</code>
Depurador	<code>agent/debugger.js</code>	Diagnostica causa raiz sem alterar código	<code>healer</code>
Curador	<code>agent/healer.js</code>	Aplica patch com base no diagnóstico	<code>qa</code> (sucesso, fecha o loop) ou <code>healer</code> de novo (até 3x) ou <code>devops</code> (desiste)
DevOps	<code>agent/devops.js</code> via <code>stages/devopsLoadStage.js</code>	Sobe sandbox Docker, injeta caos, mede desempenho	<code>deploy</code>
Deploy	<code>agent/devops.js:deploy()</code>	Publica local (Docker) ou mobile (Simulador/Emulador/macOS/Windows)	<code>human</code>
Teste Humano	<code>agent/human.js</code> / <code>lib/mobileHumanTest.js</code>	Executa fluxo real de usuário contra o deploy	<code>prodReady</code> se passou; <code>userFix</code> se falhou
Corretor	<code>agent/userFix.js</code>	Corrige com base no relato humano ou de usuário	<code>qa</code> (reabre o ciclo)
Checklist de Produção	<code>lib/productionChecklist.js</code>	Confere/completa artefatos de produção, publica release	<code>report</code> se pronto; <code>userFix</code> se não
Relatório	<code>agent/reporter.js</code>	Gera PDF final consolidado	fim — <code>status: 'completed'</code>

No **modo validate** (validar um projeto já existente no workspace, em vez de gerar do zero), o Arquiteto e o Codificador são pulados — o carregamento é feito por `lib/projectLoader.js` e o fluxo entra direto no gate `qa`, idêntico ao modo `forge` a partir daí.

Retries: até **3 tentativas de cura** (`healingAttempts`) no ciclo QA→Segurança→Depurador→Curador; na última, o Curador troca de provedor de LLM automaticamente (`escalateProvider: true`). O ciclo Teste Humano→Corretor não tem teto rígido de tentativas, mas troca de provedor a partir da 3ª (`maxUserFixAttemptsBeforeEscalate`).

Um relato de usuário pode ser injetado a qualquer momento fora do fluxo linear via `orchestrator.queueUserReport(message)` (usado pelo campo de chat do Terminal), desde que a run não esteja concluída/cancelada/falhada nem executando no momento.

13. API REST completa

Autenticação Bearer (`Authorization: Bearer <token>`) em toda rota `/api/*` exceto `/api/health` e `/api/llm/status` . Rate limiting global (600 req/5min) e um limite mais apertado para tentativas de autenticação inválidas (30/15min).

Método	Rota	RBAC	Descrição
GET	<code>/api/health</code>	público	Prontidão (DB, Docker, Ollama, LLM)
GET	<code>/api/llm/status</code>	público	Probe de um provedor LLM
GET	<code>/api/llm/usage</code>	autenticado	Uso agregado por período e cooldowns
POST	<code>/api/llm/cooldown/:provider/clear</code>	autenticado	Limpa cooldown manual de um provedor
GET	<code>/api/ops/health</code>	autenticado	Saúde operacional (falhas seguidas, runs travadas)
GET	<code>/api/ops/dogfood</code>	autenticado	Status do agendamento de dogfooding + última run
POST	<code>/api/audit/run</code>	autenticado	Dispara auditoria independente (Semgrep + npm audit)
GET	<code>/api/audit/runs</code>	autenticado	Lista auditorias
GET	<code>/api/audit/runs/:id</code>	autenticado	Detalhe de uma auditoria
GET/POST	<code>/api/preferences</code>	autenticado	Regras de estilo do Engenheiro Sênior
POST	<code>/api/preferences/reset-senior</code>	autenticado	Restaura regras padrão
GET/POST/PATCH/DELETE	<code>/api/projects[:id]</code>	autenticado	CRUD de projetos
POST	<code>/api/projects/ensure</code>	autenticado	Cria pasta+projeto se necessário
GET	<code>/api/runs</code>	autenticado	Lista runs (limite ≤ 200)
GET	<code>/api/runs/stats/reliability</code>	autenticado	Estatísticas de confiabilidade
GET	<code>/api/runs/:id[/export /report.pdf]</code>	autenticado	Detalhe, exportação e PDF de uma run
GET	<code>/api/runs/:id/files/:filePath/versions</code>	autenticado	Histórico de versões de um arquivo
GET	<code>/api/ollama/models</code>	autenticado	Modelos disponíveis no Ollama local
GET	<code>/api/agent/status</code>	autenticado	Estado atual do orquestrador

GET	<code>/api/team / /api/team/me / /api/team/board</code>	autenticado	Equipe, identidade própria, quadro de runs
POST	<code>/api/team/members</code>	admin	Cria membro
POST	<code>/api/team/members/:id/deactivate</code>	admin	Desativa membro
GET	<code>/api/services/status</code>	autenticado	Status do serviço/watchdog
POST	<code>/api/services/:action</code>	admin/lead/sre	Controla o serviço (start/stop/restart/watch)
POST	<code>/api/fs/browse / /api/fs/mkdir</code>	autenticado	Navegação de workspace (sem path traversal)
GET	<code>/api/workspace</code>	autenticado	Raiz do workspace configurada
POST	<code>/api/agent/run</code>	não-viewer	Inicia (ou enfileira) uma forja
POST	<code>/api/agent/validate</code>	não-viewer	Inicia (ou enfileira) validação de projeto existente
POST	<code>/api/agent/approve</code>	por etapa (ver seção 17)	Aprova e avança a etapa pendente
POST	<code>/api/agent/cancel</code>	não-viewer	Cancela a run atual
POST	<code>/api/agent/user-report</code>	não-viewer	Envia relato ao Corretor
GET	<code>/api/docker/status</code>	autenticado	Status do daemon Docker

Em produção, o mesmo processo também serve o build estático do frontend (`frontend/dist`), com fallback de SPA para qualquer rota que não seja `/api/*`.

14. WebSocket e eventos ao vivo

Conexão autenticada (`authenticateWs`); ao conectar, o servidor envia imediatamente um evento `sync-state` com o estado atual do orquestrador — importante pra UI recuperar o estado certo mesmo se a run já estava em andamento antes da conexão abrir. Eventos subsequentes incluem `task-awaiting-approval`, `agent-finished`, `task-cancelled`, `task-failed`, entre outros ligados a cada etapa.

15. Banco de dados

SQLite via `better-sqlite3` (caminho em `FORJA_DB_PATH`, padrão `data/forja.db`). Persiste: projetos, execuções (runs) com todo o estado necessário para sobreviver a um restart do processo no meio de uma run (ADR-030), eventos de log por run, versões históricas de cada arquivo gerado, preferências (regras de estilo), e membros de equipe (token com hash SHA-256, nunca em texto puro). Backup via `lib/dbBackup.js` (API de backup online do SQLite, não uma cópia de arquivo bruta) — ver seção 18 para as variáveis de retenção.

16. Provedores de LLM e fallback

Suportados: `gemini`, `claude`, `openai` (e compatíveis via `OPENAI_BASE_URL`), `ollama` (local) e `cursor` (Cursor Agent CLI, opt-in, nunca escolhido automaticamente).

Seleção do provedor primário: respeita escolha explícita da UI; senão usa o default (`FORJA_LLM_PROVIDER`) — mas se esse default estiver em cooldown por erro de billing recente, troca automaticamente para o provedor cloud configurado com menor uso de tokens hoje.

Provedor de revisão sênior: deliberadamente diferente do provedor primário quando há alternativa (reduz erro correlacionado — mesmo raciocínio de dois provedores diferentes tende a discordar menos por acaso).

Fallback em erro: erro de billing → cooldown do provedor + prioriza outros provedores cloud pagos antes do Ollama; erro genérico (rede/timeout) → tenta Ollama primeiro (mais confiável nesse cenário), depois os cloud configurados. Sem nenhum provedor disponível e `FORJA_ALLOW MOCKS=false` (padrão), a run falha explicitamente — nunca finge sucesso.

Cada provedor tem um modelo "premium" (usado nas etapas que geram entregável) e um modelo "economy" (usado só na camada opcional de revisão sênior).

17. RBAC e equipe

Papéis: `admin`, `lead`, `qa`, `sre`, `member`, `viewer`. Tabela de aprovação por etapa (`STAGE_ROLES`, `backend/lib/rbac.js`):

Etapa	Papéis que podem aprovar
coder, debugger, healer, userFix	admin, lead
qa, security, human	admin, qa, lead
devops, deploy, prodReady, report	admin, sre, lead

O admin é uma identidade fixa derivada do `FORJA_API_TOKEN` (comparação em tempo constante). Demais membros são persistidos no SQLite com token com hash; sem `FORJA_TEAM_JSON`, o sistema semeia 3 membros de bootstrap (Lead/QA/SRE) com token derivado do hash do token admin.

18. Variáveis de ambiente

Variável	Padrão	Descrição
<code>HOST</code>	<code>127.0.0.1</code>	Host de bind
<code>PORT</code>	<code>3001</code>	Porta do servidor
<code>CORS_ORIGIN</code>	<code>http://localhost:5173</code>	Origem CORS permitida
<code>FORJA_API_TOKEN</code>	gerado se vazio	Token do admin; ≥24 chars obrigatório em produção
<code>FORJA_WORKSPACE_ROOT</code>	<code>backend/dev</code>	Raiz dos projetos forjados/validados
<code>FORJA_DB_PATH</code>	<code>data/forja.db</code>	Caminho do SQLite

FORJA_BACKUP_DIR / FORJA_BACKUP_RETENTION	data/backups / 14	Pasta e retenção de backups
FORJA_LLM_PROVIDER	ollama	Provedor default
GEMINI_API_KEY / GEMINI_MODEL	— / gemini-3.6-flash	Config Gemini
OPENAI_API_KEY / OPENAI_BASE_URL / OPENAI_MODEL	— / api oficial / gpt-4.1	Config OpenAI (ou compatível)
ANTHROPIC_API_KEY / ANTHROPIC_MODEL	— / claude-sonnet-4-20250514	Config Claude
*_MODEL_ECONOMY	por provedor	Modelo econômico só para revisão sênior
OLLAMA_BASE_URL / OLLAMA_DEFAULT_MODEL	http://127.0.0.1:11434 / qwen2.5-coder:7b	Config Ollama
CURSOR_API_KEY / CURSOR_MODEL / CURSOR_AGENT_BIN	— / auto / cursor-agent	Config Cursor Agent (opt-in)
FORJA_REQUIRE_DOCKER	true	Exige Docker (obrigatório em produção)
FORJA_ALLOW MOCKS	false	Permite resposta simulada em vez de LLM real (proibido em produção)
FORJA_ALLOW_PUBLIC_BIND	false	Permite bind fora de loopback em produção
FORJA_LLM_TIMEOUT_MS / FORJA_LLM_RETRIES	300000 / 2	Timeout e retries de chamada LLM
FORJA_SANDBOX_PORT / FORJA_DEPLOY_PORT / FORJA_STAGING_PORT	4000 / 5100 / 5200	Portas host
FORJA_REQUIRE_GIT_PR	false	Exige PR em vez de commit direto no release
FORJA_TEAM_JSON	—	Membros de equipe pré-cadastrados (JSON)
FORJA_AUDIT_SCHEDULE_HOURS	0 (desligado)	Intervalo de auditoria independente agendada
FORJA_RUN_BUDGET_USD	0 (desligado)	Teto de gasto estimado por run
FORJA_WATCH_UNHEALTHY_THRESHOLD	3	Falhas seguidas antes do watchdog reiniciar
NODE_ENV	development	Ativa validações extras de produção

Em produção: token ≥ 24 chars, mocks proibidos, Docker obrigatório, host precisa ser loopback (a menos que `FORJA_ALLOW_PUBLIC_BIND`), e o provedor default precisa ter chave configurada (exceto ollama/cursor).

19. Scripts npm

Comando	O que faz
<code>npm run dev</code>	Backend (nodemon) + frontend (vite) em paralelo
<code>npm run dev:stable</code>	Backend sem nodemon + frontend em dev
<code>npm run start:local-prod</code>	Build do frontend + checagem de config + backend em produção
<code>npm run check:local-prod</code>	Só a checagem de config de produção
<code>npm run service:status start stop restart watch</code>	Controle do serviço persistente (watch = watchdog de health)
<code>npm test</code> / <code>test:backend</code> / <code>test:frontend</code>	Suítes de teste
<code>npm run build</code>	Build de produção do frontend
<code>npm run audit:self</code> / <code>audit:project</code>	Auditoria independente (Semgrep + npm audit)
<code>npm run backup:db</code>	Backup do SQLite
<code>npm run dogfood</code>	Dispara uma rodada de dogfooding manualmente

20. Testes

Backend: `node --test --test-concurrency=1 test/*.test.js` — runner nativo do Node, execução serial (cada arquivo isola seu próprio banco/porta). Estado atual: **350 testes, 127 suítes, 0 falhas**. Frontend: Vitest + Testing Library. Estado atual: **143 testes, 21 arquivos, 0 falhas**.

Convenções estabelecidas no projeto: testes de integração HTTP sobem o `server.js` real (`httpIntegration.test.js`), não mocks de rota; módulos com dependência de processo externo (Docker, Appium, GitHub Actions) recebem funções injetáveis para teste determinístico, mas o padrão do projeto favorece verificação ao vivo contra a ferramenta real antes de considerar uma integração concluída — documentado em ADRs quando aplicável.

21. Deploy multiplataforma

`agent/stages/deployStage.js` chama `devops.deploy()`, que detecta o tipo de projeto (`lib/projectType.js`):

- **Web**: Docker local, escrevendo `Dockerfile/.dockerignore/.env.example` padrão se ausentes.
- **Mobile (Expo/RN)**: sem servidor HTTP — publica em até 4 alvos independentes: Simulador iOS e Emulador Android (sempre), macOS Catalyst e Windows/GitHub Actions (só se já configurados no projeto). O resultado consolidado (`deployResult.targets`) alimenta a decisão de contra quais apps rodar o Teste Humano via Appium.

22. Dogfooding automático (implementação)

`scripts/dogfood-forge.js` dispara uma forja real via API (`POST /api/agent/run`), faz polling em `GET /api/agent/status` , aprova cada gate sozinho (`POST /api/agent/approve`) até um estado terminal ou um teto de tempo, e grava um relatório em `backend/data/dogfood-reports/<timestamp>.{json,md}` . Nunca inicia se já existe uma run em andamento. Agendado via **crontab do macOS** (não uma ferramenta de agendamento de sessão de IA, que não seria persistente) — toda segunda às 6h, sobe o serviço se necessário e roda o script. O estado (agendado? última run?) é exposto por `GET /api/ops/dogfood` (`backend/lib/dogfoodStatus.js` , que lê o crontab real e o último relatório em disco) e consumido pela UI (ponto de status no cabeçalho + cartão na aba Auditoria). Ver ADR-035.

23. Decisões arquiteturais (ADRs)

Lista completa em `docs/adr/README.md` ; resumo de uma linha cada:

1. Extrair estágios do orchestrator em módulos separados
2. Decompor `App.tsx` e `useForjaApp.ts`
3. Chaos engineering real via API do Docker, com fallback simulado
4. TypeScript strict mode e eliminação de `any`
5. Endurecimento de segurança pontual (rate limit, tokens, purge)
6. Watchdog exige falhas seguidas antes de reiniciar
7. Cursor Agent como provedor de LLM opt-in, isolado em cwd descartável
8. Prompt caching para Claude; sem mudança para os demais provedores
9. CI no Node 22 + isolamento de DB por arquivo de teste
10. Modelo econômico na camada de revisão sênior, entregável intocado
11. Diversidade de provedor na revisão sênior + scanner determinístico de segredos
12. Instrumentação de confiabilidade medida por run
13. Card de confiabilidade na UI + escalada de provedor na última cura
14. Suporte a projetos mobile Expo/React Native (QA nativo + deploy no Simulador)
15. Fallback prioriza outro provedor cloud quando a falha é de billing
16. Corretor manda só os arquivos relevantes, não o codebase inteiro
17. Uso equilibrado entre provedores via dado real (não saldo de crédito)
18. Deploy mobile também para macOS (Catalyst) e Windows (GitHub Actions)
19. Pente fino: 2 críticos + 4 altos corrigidos numa auditoria dedicada
20. Restante do pente fino: os ~13 achados médios/baixos
21. Auditoria independente (Semgrep + npm audit), separada do pipeline
22. Teste humano com navegador real (Playwright)
23. Aba "Auditoria" na UI
24. Teto de orçamento estimado por run
25. Papel "viewer" + fecha o RBAC não-enforced em run/cancel/relato
26. Corretor escala de provedor + consistência de sistema de módulos na constituição
27. Fecha as lacunas de cobertura de teste no frontend
28. UI de gestão de equipe (criar e desativar membros)

- 29. Teste humano real no Simulador via Appium/XCUITest
- 30. Segurança de restart sistemática + esquema canônico de estado de run
- 31. Deploy e teste humano no emulador Android
- 32. Backup do SQLite + procedimento de restore documentado
- 33. Endpoint de saúde operacional agregada
- 34. QA parava de confiar em código correto por causa de um acoplamento com os mocks
- 35. Dogfooding automático e agendado (script + crontab do macOS)

24. Como estender o pipeline

Para adicionar um novo estágio: (1) crie o módulo do agente em `backend/agent/` ; (2) crie o wrapper em `backend/agent/stages/` que decide, ao final, para qual etapa chamar `pauseForApproval` ; (3) registre o rótulo em `STAGE_LABELS` (`orchestrator.js`); (4) adicione a etapa em `STAGE_ROLES` (`lib/rbac.js`) definindo quem pode aprová-la; (5) se o estágio grava um campo novo em `currentTask` / `savedConfig` que outro código lê diretamente, garanta que ele sobrevive a um restart no meio da run (ver checklist do ADR-030 em `docs/adr/README.md` e o molde de teste em `backend/test/restartSafety.test.js`); (6) escreva testes unitários do módulo isolado (com dependências injetáveis para chamadas externas) e, se integrar uma ferramenta externa nova, uma seção "Verificação ao vivo" no ADR correspondente, testando contra a ferramenta real, não só mock.